Evaluación Integrada de Desempeño N°1

Análisis y Modelamiento de Secciones Cónicas y Funciones por Tramos a partir del RUT

1. Descripción

El siguiente documento sistematiza la información necesaria para desarrollar la Evaluación Integrada de Desempeño del curso MAT1186. Este proyecto representa un 25 % de la calificación final y busca integrar contenidos matemáticos del curso con programación, razonamiento lógico y trabajo colaborativo.

1. Competencias genéricas

Nombre: Actuación ética

Definición: Actúa con sentido ético sustentando su discernimiento en valores de justicia, bien común y dignidad del ser humano, entendiendo la profesión como un servicio que da respuesta a las necesidades de las personas, la comunidad y el medioambiente.

Nombre: Aprendizaje autónomo

Definición: Se responsabiliza de su propio aprendizaje, lo que le lleva a utilizar procesos cognitivos y metacognitivos para aprender de forma estratégica y flexible, en función del objetivo de aprendizaje.

2. Competencias específicas

Nombre: Aplica ciencias de la Ingeniería

Definición: Implementa modelos matemáticos y otros propios de la ingeniería y de las ciencias básicas, empleando habilidades de razonamiento lógico deductivo para abordar problemas de análisis y diseño de sistemas tecnológicos basados en software, ligados a especialidades de la ingeniería.

3. Resultado de aprendizaje (RA1)

Implementa, conoce y maneja los elementos y conceptos de geometría analítica y límites, aplicando la competencia de aprendizaje autónomo.

2. Contexto del proyecto

En distintos problemas de modelamiento matemático y computacional, las ecuaciones de segundo grado en dos variables permiten representar trayectorias, formas geométricas y comportamientos espaciales. En este proyecto, cada grupo deberá construir y analizar una sección cónica a partir de los datos extraídos desde un RUT chileno válido, integrando criterios matemáticos y computacionales en un programa desarrollado en Python.

2.1. Objetivo del proyecto

El objetivo principal del proyecto es desarrollar una aplicación en Python que permita validar un RUT chileno, construir automáticamente una ecuación general de segundo grado a partir de sus dígitos, clasificar la cónica obtenida, transformarla cuando corresponda a su forma canónica y graficarla en el plano cartesiano. Además, el sistema deberá incorporar un módulo de análisis de funciones por tramos generadas a partir del mismo RUT, con el propósito de estudiar límites laterales, continuidad y tipos de discontinuidad mediante una lógica algorítmica implementada manualmente.

Además, se busca que el trabajo sea desarrollado de manera profesional, con una interfaz intuitiva, clara y visualmente cuidada, adecuada para la interacción del usuario y para la defensa oral del proyecto. Cada grupo podrá utilizar el framework web o la librería de interfaz que estime conveniente, siempre que el resultado final sea funcional, coherente y bien diseñado.

El desarrollo deberá realizarse con adecuada organización del código, distribución real de tareas entre los integrantes del grupo y uso de GitHub como herramienta de control de versiones y evidencia de participación. Asimismo, se espera que cada grupo trabaje bajo una estructura interna de organización, con un líder designado y un código de ética propio que permita regular la colaboración y resolver eventuales conflictos de trabajo.

3. Fases de trabajo

3.1. Fase 1: Fundamento matemático

Sea el RUT del tipo:

$$d_1d_2d_3d_4d_5d_6d_7d_8 - \mathrm{DV}$$

donde $d_1, d_2, \ldots, d_8$ representan los ocho dígitos del cuerpo del RUT y DV es el dígito verificador.

El programa deberá verificar primero que el RUT ingresado sea válido utilizando el algoritmo oficial del módulo 11. Esta validación no debe ser superficial, sino que debe estar implementada correctamente en el código, mostrando el procedimiento correspondiente para determinar si el dígito verificador coincide con el valor esperado.

Para efectos del modelo matemático, se define una variable auxiliar v asociada al dígito verificador:

$$v = \begin{cases} 10, & \text{si DV} = \text{K} \\ 11, & \text{si DV} = 0 \\ \text{DV}, & \text{si DV es un dígito entre 1 y 9} \end{cases}$$

Luego, a partir de los dígitos del RUT, se construirá una ecuación general de segundo grado de la forma:

$$Ax^2 + By^2 + Cx + Dy + E = 0$$

donde:

$$A = \frac{d_1 + d_2}{v}, \qquad B = \frac{d_3 + d_4}{v}, \qquad C = -(d_5 + d_6), \qquad D = -(d_7 + d_8), \qquad E = d_1 + d_3 + d_5 + d_7$$

Ajustes para obtener distintas cónicas

Con el objetivo de asegurar variedad en los resultados, se aplicarán además las siguientes reglas:

- Si d_8 es impar, entonces se reemplaza B por $-B$. Esto permite la aparición de hipérbolas.
- Si $d_1 = d_2$, entonces se impone $B = A$. Esto permite la aparición de circunferencias.
- Si $d_5 + d_6$ es múltiplo de 3, entonces se generará una parábola, diferenciando su orientación de la siguiente manera:
  - Si d_7 es par, se define $B = 0$, obteniendo una parábola de eje vertical.
  - Si d_7 es impar, se define $A = 0$, obteniendo una parábola de eje horizontal.

Clasificación de la cónica

Una vez construida la ecuación general, el programa deberá determinar automáticamente el tipo de cónica de acuerdo con los siguientes criterios:

- **Circunferencia:** si $A = B$ y ambos son distintos de cero.
- **Elipse:** si A y B tienen el mismo signo y además $A \neq B$.
- **Hipérbola:** si A y B tienen signos opuestos.
- **Parábola:** si exactamente uno de los coeficientes A o B es igual a cero.

Además de clasificar la cónica, el programa deberá mostrar el desarrollo matemático paso a paso de la transformación de la ecuación general a la forma canónica, justificando claramente cada operación algebraica realizada. Del mismo modo, deberá mostrar el procedimiento inverso, es decir, cómo a partir de la forma canónica se puede regresar a la ecuación general. No basta con mostrar el resultado final: el objetivo es que el estudiante evidencie comprensión completa del procedimiento matemático.

3.2. Fase 2: Desarrollo del programa

Cada grupo deberá desarrollar una aplicación en Python que cumpla, como mínimo, con los siguientes requerimientos:

1. Ingresar y validar un RUT chileno real mediante la implementación correcta del algoritmo oficial del dígito verificador.
2. Mostrar, paso a paso, el procedimiento utilizado para validar el RUT ingresado.
3. Extraer correctamente los dígitos del RUT.
4. Mostrar, paso a paso, cómo se construye la ecuación general según las condiciones dadas.
5. Construir la ecuación general asociada al RUT.
6. Determinar de manera automática si la ecuación representa una circunferencia, parábola, elipse o hipérbola.
7. Mostrar en pantalla la ecuación general obtenida.
8. Mostrar, cuando corresponda, la forma canónica de la cónica.
9. Resolver y mostrar paso a paso la transformación desde la ecuación general a la forma canónica.
10. Resolver y mostrar paso a paso el procedimiento inverso, desde la forma canónica a la ecuación general.
11. Graficar correctamente la cónica obtenida en el plano cartesiano.
12. Presentar una interfaz intuitiva, ordenada y visualmente adecuada, evitando soluciones incompletas, confusas o improvisadas.
13. Incorporar el módulo de análisis de funciones por tramos generado a partir del RUT, asegurando coherencia entre los distintos componentes del sistema.

Se permitirá el uso de cualquier framework web o librería de interfaz que el grupo considere pertinente para el desarrollo visual de la aplicación.

Sin embargo, se prohíbe estrictamente el uso de librerías matemáticas o de álgebra computacional tales como `numpy`, `math`, `sympy` u otras equivalentes. Todos los cálculos requeridos en el proyecto deberán ser implementados manualmente en el código, con el fin de evidenciar comprensión real de los procedimientos matemáticos involucrados.

3.3. Fase 3: Desarrollo profesional del código

El trabajo no debe resolverse como un único archivo desordenado. Se espera una implementación clara, modular y profesional.

Por lo tanto, el proyecto deberá considerar, como mínimo, los siguientes aspectos:

1. Uso obligatorio de GitHub como repositorio del proyecto.
2. Existencia de evidencia de participación de todos los integrantes del grupo mediante commits u otros aportes verificables, minimo unos 3 commits por integrante.
3. Organización modular del código, separando responsabilidades en distintos archivos cuando corresponda.
4. Uso de nombres de variables, funciones y archivos que sean claros y coherentes.
5. Se espera que no haya una cantidad excesiva de comentarios, sino solamente los necesarios y pertinentes.
6. Manejo básico de errores de entrada, especialmente para casos de RUT mal ingresado o inválido.
7. Cada grupo deberá estar conformado por exactamente 3 integrantes, sin excepciones. No se aceptarán grupos de 2 ni de 4 estudiantes.
8. La calificación del proyecto será grupal.
9. Cada grupo deberá designar formalmente un líder de grupo.
10. Cada grupo deberá redactar y presentar un código de ética interno, cuyo propósito será establecer criterios de trabajo, responsabilidades y mecanismos de resolución de conflictos entre sus integrantes.

3.4. Fase 4: Casos de prueba y visualización gráfica

Cada grupo deberá probar su programa con distintos RUTs válidos, de manera que pueda evidenciar el funcionamiento del sistema y la correcta clasificación de distintas secciones cónicas.

Se espera que el grupo sea capaz de presentar ejemplos que permitan observar las cuatro cónicas estudiadas: circunferencia, parábola, elipse e hipérbola.

No se aceptarán RUTs inválidos, inventados o sin verificación correcta del dígito verificador.

La graficación debe ser matemáticamente correcta y coherente con la ecuación obtenida. Además, la interfaz deberá incluir entradas o campos de texto inicialmente vacíos, de manera que durante la defensa oral el o los estudiantes puedan completar manualmente la ubicación de elementos relevantes de la cónica según corresponda.

Entre estos elementos pueden considerarse, según el tipo de cónica:

- centro,
- vértices,
- focos,
- eje mayor o eje transverso,

- eje menor o eje conjugado,
- directriz, en caso de corresponder.

El objetivo de esta funcionalidad es que, al momento de la defensa, el estudiante no solo observe la gráfica, sino que también sea capaz de identificar correctamente los elementos geométricos principales de la cónica y ubicarlos de acuerdo con el análisis matemático realizado.

Asimismo, la interfaz no debe limitarse a cumplir de forma mínima con la funcionalidad. Se espera una presentación intuitiva, ordenada, consistente y visualmente agradable, de manera que la interacción con el programa sea clara tanto para el usuario como para el equipo docente durante la evaluación.

3.5. Fase 5: Defensa oral

La evaluación final contemplará una defensa oral grupal. En esta instancia, se realizarán preguntas centradas principalmente en el código desarrollado, en las decisiones de implementación adoptadas y en la comprensión matemática del problema.

Las preguntas podrán abordar, entre otros aspectos, la validación del RUT, la construcción de coeficientes, la clasificación de la cónica, la lógica de graficación, la transformación de la forma general a la canónica, el procedimiento inverso, la estructura del programa y el aporte individual de cada integrante.

Todos los integrantes del grupo deben estar en condiciones de explicar su trabajo. Además, la persona que deberá responder o exponer partes específicas durante la defensa será seleccionada bajo condiciones especiales completamente aleatorias, las cuales serán dadas a conocer únicamente en el momento de la evaluación.

Durante la defensa se podrá solicitar que el estudiante seleccionado complete en la interfaz algunos de los campos asociados a los elementos de la cónica representada, con el fin de validar su comprensión matemática sobre centro, focos, vértices y otros elementos relevantes según el caso.

Si durante la defensa se evidencia que el grupo no comprende el funcionamiento general del programa, no puede justificar las decisiones tomadas o no demuestra dominio matemático suficiente del problema, ello afectará la calificación grupal.

3.6. Fase 6: Análisis de funciones por tramos generadas a partir del RUT

El proyecto deberá incorporar un módulo adicional dentro de la aplicación, enfocado en el análisis del comportamiento de funciones generadas algorítmicamente a partir de los dígitos del RUT.

A diferencia de una calculadora de límites tradicional, este módulo no se centrará en resolver funciones arbitrarias ingresadas libremente por el usuario, sino en analizar funciones construidas automáticamente por el sistema. El objetivo es que el estudiante pueda relacionar el comportamiento computacional de una función con su interpretación matemática mediante límites laterales, continuidad y discontinuidades.

Construcción de la función a partir del RUT

Sea el RUT del tipo:

$$d_1d_2d_3d_4d_5d_6d_7d_8 - \text{DV}$$

A partir de sus dígitos, el sistema deberá construir una función definida por tramos. Para evitar casos excesivamente simples, el punto de análisis principal se definirá como:

$$a = d_3$$

El programa deberá generar una función por tramos de la forma:

$$f(x) = \begin{cases} f_1(x), & \text{si } x < a \\ f_2(x), & \text{si } x \geq a \end{cases}$$

donde cada tramo se construirá utilizando los dígitos del RUT. Como mínimo, el sistema deberá implementar los siguientes tipos de funciones generadas:

Caso 1: Discontinuidad removible

Se construirá una función racional con un factor común que se anula en el punto de análisis:

$$f_1(x) = \frac{(x-a)(x+d_1)}{x-a}$$

En este caso, el programa deberá simplificar manualmente la expresión y reconocer que la función original no está definida en $x = a$, aunque el límite puede existir.

$$\lim_{x \to a} \frac{(x-a)(x+d_1)}{x-a} = a + d_1$$

Este caso permite analizar una discontinuidad removible.

Caso 2: Discontinuidad de salto

Se construirán dos tramos con comportamientos distintos al acercarse al punto a:

$$f(x) = \begin{cases} x + d_2, & \text{si } x < a \\ x + d_4, & \text{si } x \geq a \end{cases}$$

El programa deberá calcular:

$$\lim_{x \to a^-} f(x) = a + d_2$$
$$\lim_{x \to a^+} f(x) = a + d_4$$

Si ambos valores son distintos, el sistema deberá concluir que el límite no existe y clasificar la discontinuidad como discontinuidad de salto.

Caso 3: Discontinuidad infinita

Se construirá una función racional cuyo denominador se anula en el punto de análisis:

$$f(x) = \frac{d_5 + 1}{x - a}$$

El programa deberá analizar el comportamiento por izquierda y por derecha, identificando si la función crece o decrece sin límite al aproximarse a a. Este caso permite reconocer una discontinuidad infinita y la presencia de una asíntota vertical en:

$$x = a$$

Regla de selección del caso

El sistema deberá seleccionar automáticamente el tipo de función a generar a partir del RUT, utilizando la siguiente regla:

- Si d_8 es múltiplo de 3, se generará el caso de discontinuidad removible.
- Si d_8 deja residuo 1 al dividirse por 3, se generará el caso de discontinuidad de salto.
- Si d_8 deja residuo 2 al dividirse por 3, se generará el caso de discontinuidad infinita.

Esta regla deberá mostrarse en la interfaz, indicando claramente por qué el programa generó ese tipo de función a partir del RUT ingresado.

Requerimientos del sistema

El programa deberá ser capaz de identificar automáticamente los puntos críticos de la función, tales como puntos donde cambia el tramo, puntos donde el denominador se anula y puntos donde la función no está definida.

Para cada punto crítico, el sistema deberá calcular los límites laterales:

$$\lim_{x \to a^-} f(x)$$

$$\lim_{x \to a^+} f(x)$$

Luego deberá determinar si el límite existe, comparando ambos resultados. Además, deberá analizar la continuidad de la función en el punto, verificando si el límite existe, si la función está definida en dicho punto y si el valor de la función coincide con el valor del límite.

El programa también deberá clasificar el tipo de discontinuidad presente, pudiendo ser discontinuidad removible, discontinuidad de salto o discontinuidad infinita. No basta con mostrar el nombre de la discontinuidad: el sistema deberá justificar matemáticamente por qué corresponde a ese caso.

Evidencia computacional

El sistema deberá complementar el análisis matemático mediante evidencia numérica. Para ello, deberá mostrar una tabla de valores cercanos al punto de análisis a, considerando valores por izquierda y por derecha.

Por ejemplo, si el punto de análisis es a, el programa puede evaluar:

$$a-1, \quad a-0{,}1, \quad a-0{,}01, \quad a-0{,}001$$

$$a+0{,}001, \quad a+0{,}01, \quad a+0{,}1, \quad a+1$$

Esta tabla deberá permitir observar el comportamiento de la función al aproximarse al punto crítico. El objetivo no es reemplazar el análisis matemático, sino contrastar la evidencia computacional con la conclusión obtenida mediante límites.

Interfaz gráfica del módulo

La interfaz gráfica del módulo de límites deberá ser clara, ordenada y coherente con el resto de la aplicación. Además de mostrar la función generada, la tabla de valores, la gráfica y el procedimiento matemático, deberá incluir campos de entrada inicialmente vacíos para que el estudiante los complete durante la defensa oral.

Como mínimo, la interfaz deberá incluir campos vacíos para ingresar:

- el límite por la izquierda,
- el límite por la derecha,
- la conclusión sobre la existencia del límite,
- el valor de la función en el punto, si existe,
- la conclusión sobre continuidad,
- el tipo de discontinuidad,
- una justificación escrita del comportamiento observado.

Estos campos no deben venir completados automáticamente. La finalidad es que el estudiante interprete la información entregada por el sistema y demuestre comprensión del concepto de límite. Durante la defensa, se podrá solicitar que el estudiante complete estos campos manualmente y explique sus respuestas.

Representación gráfica

El programa deberá graficar la función completa y mostrar con claridad el comportamiento en torno al punto crítico. La gráfica deberá evidenciar visualmente si existe continuidad, salto, ruptura o crecimiento infinito.

En caso de discontinuidad removible, la gráfica deberá representar que existe un punto no definido en $x = a$. En caso de discontinuidad de salto, deberá mostrarse la diferencia entre el comportamiento por izquierda y por derecha. En caso de discontinuidad infinita, deberá evidenciarse la tendencia de la función hacia valores cada vez mayores o menores cerca del punto crítico.

Restricciones de implementación

El desarrollo de este módulo deberá cumplir estrictamente con las siguientes condiciones:

- Se prohíbe estrictamente el uso de librerías matemáticas, de álgebra computacional o de cálculo simbólico que faciliten la resolución automática del problema.
- Queda prohibido el uso de librerías tales como `numpy`, `math`, `sympy`, `scipy`, `pandas` u otras equivalentes para realizar cálculos matemáticos, simplificaciones algebraicas, evaluación automática de funciones, límites o graficación basada en procesamiento matemático avanzado.
- Todos los cálculos deberán ser implementados manualmente en el código.
- La simplificación de expresiones, el cálculo de límites laterales, la clasificación de discontinuidades y la generación de tablas de valores deberán programarse mediante lógica propia.
- No se evaluará únicamente el resultado final, sino también el procedimiento, la lógica utilizada y la capacidad del estudiante para explicar el funcionamiento del algoritmo.

Se permitirá el uso de librerías o herramientas orientadas exclusivamente a la construcción de la interfaz gráfica, siempre que no resuelvan automáticamente los cálculos matemáticos del proyecto.

Uso en la defensa

Durante la defensa, se podrá solicitar al estudiante que explique el comportamiento de la función en un punto específico, justifique la existencia o no existencia del límite, complete los campos vacíos de la interfaz, clasifique correctamente la discontinuidad, interprete la gráfica generada y relacione la evidencia computacional con el análisis matemático.

El estudiante deberá ser capaz de explicar cómo se generó la función a partir del RUT, por qué se produjo determinado tipo de discontinuidad y cómo el programa llegó a sus conclusiones.

El objetivo de este módulo es evaluar la capacidad del estudiante para relacionar algoritmos, análisis matemático y comprensión conceptual del límite.

3.7. Conformación de grupos

El proyecto deberá ser desarrollado en grupos de exactamente 3 integrantes, sin excepciones. No se aceptarán grupos de 2 ni de 4 personas.

Cada grupo deberá definir un líder, quien tendrá la responsabilidad de coordinar el trabajo, canalizar la comunicación interna y velar por el cumplimiento de los acuerdos del equipo.

Además, cada grupo deberá elaborar un código de ética interno, orientado a regular la forma de trabajo, la distribución de responsabilidades y la resolución de conflictos entre sus integrantes. Este documento deberá ser coherente con el desarrollo del proyecto y con el compromiso asumido por cada miembro del equipo.

4. Entrega y evaluación

El proyecto deberá ser entregado en la fecha informada por el equipo docente, considerando como producto principal el código fuente desarrollado y el enlace al repositorio de GitHub correspondiente.

La evaluación considerará tanto el funcionamiento del programa como la calidad técnica de su implementación y la defensa oral realizada por el grupo.

Nota importante: El proyecto debe ser desarrollado sobre la base de RUTs válidos y con evidencia verificable de trabajo colaborativo. El repositorio deberá estar actualizado al momento de la revisión y defensa.